

Week2

命令行环境；开发环境与工具；Debugging and Profiling

姓名：赵均临 学号：25020007178

日期：2026 年 9 月 6 日

目录

1 课上实验题目	3
1.1 实验5：控制一个可清理的后台任务	3
1.1.1 实现逻辑	3
1.1.2 完整脚本 worker.sh	3
1.1.3 操作流程	3
1.1.4 核心知识点	4
1.2 实验6：语义重构与本地开发反馈	4
1.2.1 实现逻辑	4
1.2.2 初始 Python 代码	4
1.2.3 操作流程	4
1.2.4 核心知识点	5
1.3 实验7：用调试器定位归并排序缺陷	5
1.3.1 问题描述	5
1.3.2 缺陷代码分析	5
1.3.3 调试过程	6
1.3.4 修复后完整代码	6
1.3.5 测试结果	6
1.3.6 核心知识点	7
1.4 实验8：先测量，再优化慢速词频程序	7
1.4.1 原慢速代码	7
1.4.2 操作流程	7
1.4.3 优化后代码	8
1.4.4 核心知识点	8
2 练习:命令行环境	9
2.1 参数与 Globs	9
2.1.1 - 参数的作用	9
2.2 ls 命令组合参数	9

目录	2
2.3 环境变量与返回码	9
2.3.1 marco 与 polo 函数	9
2.4 循环调试脚本	10
2.5 信号与任务控制	10
2.5.1 后台任务处理	10
2.6 别名与点文件	11
2.6.1 纠错别名	11
2.7 远程机器 (SSH)	11
2.7.1 端口转发配置	11
3 练习: Debugging and Profiling	12
3.1 调试排序算法	12
3.1.1 归并排序 Bug 调试	12
3.2 反向调试	12
3.2.1 内存损坏排查	12
3.3 AddressSanitizer	12
3.3.1 内存错误检测	12
4 实验心得	13

github网址: <https://github.com/coolicer17/Systemtools>

1 课上实验题目

1.1 实验5: 控制一个可清理的后台任务

1.1.1 实现逻辑

通过Shell trap捕获终止信号, 结合作业控制实现后台任务的优雅退出, 避免强制杀死进程导致资源泄漏。

1.1.2 完整脚本 worker.sh

```
1 #!/usr/bin/env bash
2 trap 'echo "CLEAN_EXIT" >> cleanup.log; exit 0' TERM INT
3 n=0
4 while true; do echo "$n"; n=$((n+1)); sleep 1; done
```

1.1.3 操作流程

1. 加权限启动: `chmod +x worker.sh && ./worker.sh > stdout.log 2> stderr.log`

```
coolicer@ubuntu:~/q05$ chmod +x worker.sh
coolicer@ubuntu:~/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
coolicer@ubuntu:~/q05$ jobs
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
```

2. 按Ctrl+Z挂起后执行 `bg` 后台运行, `jobs` 确认状态为Running

3. 发信号终止: `kill $(jobs -p)` (禁止用kill -9)

4. SIGTERM(15)可捕获允许清理, SIGKILL(9)强制杀死不可拦截

```
coolicer@ubuntu:~/q05$ bg
[1]+ ./worker.sh > stdout.log 2> stderr.log &
coolicer@ubuntu:~/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
coolicer@ubuntu:~/q05$ # 自动拿PID发SIGTERM (kill默认就是SIGTERM/15)
coolicer@ubuntu:~/q05$ kill $(jobs -p)
coolicer@ubuntu:~/q05$
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
```

1.1.4 核心知识点

- 信号区别：SIGTERM(15)可捕获允许清理，SIGKILL(9)强制杀死不可拦截
- 作业控制：%N指代第N个作业，bg/fg/Ctrl+Z切换前后台运行状态
- trap命令：自定义信号处理动作，实现退出前的资源清理

1.2 实验6：语义重构与本地开发反馈

1.2.1 实现逻辑

利用现代代码编辑器的语言服务器协议（LSP）实现安全的语义重构，结合静态代码分析工具（ruff）和单元测试（pytest）保证代码质量与逻辑正确性。

1.2.2 初始 Python 代码

math_utils.py

```
1 def total_price(price: float, count: int) -> float:
2     return price * count
```

app.py

```
1 from math_utils import total_price
2
3 print(total_price(12.5, 4))
```

test_math_utils.py

```
1 from math_utils import total_price
2
3 def test_total_price():
4     assert total_price(12.5, 4) == 50.0
```

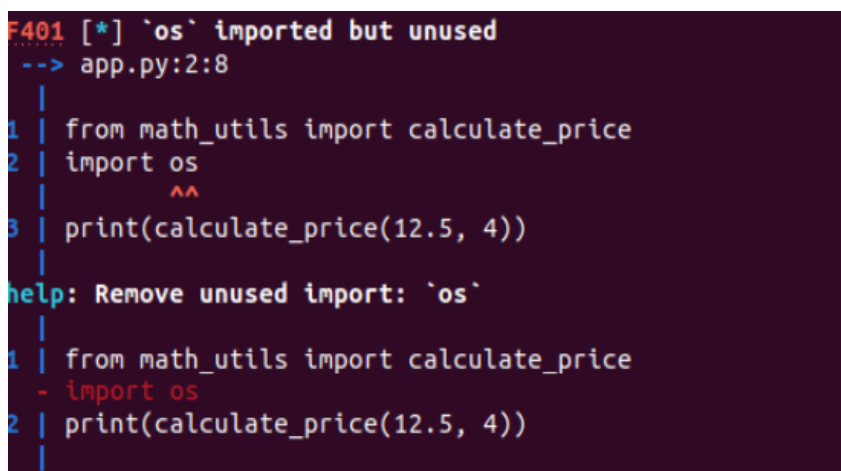
1.2.3 操作流程

1. 环境准备：在编辑器中打开目录并确认 Python 语言服务器已激活。
2. 语言特性演示：使用 F12 跳转到定义，使用 Shift+F12 查找所有引用。

3. **语义重构**: 使用重命名符号 (F2) 将 `total_price` 改为 `calculate_total`, 验证多文件引用同步更新。
4. **静态检查与测试**: 在 `app.py` 中故意加入 `import os`, 运行 `ruff check . --fix` 自动剔除未使用导入, 最后运行 `pytest` 确认测试通过。

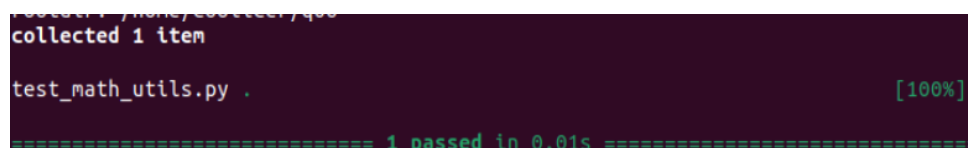
1.2.4 核心知识点

- **LSP**: 语言服务器协议提供代码跳转、引用查找和安全重命名能力。
- **Ruff**: 极速的 Python 静态代码分析工具, 可自动检测并修复未使用导入等问题。



```
F401 [*] `os` imported but unused
--> app.py:2:8
1 | from math_utils import calculate_price
2 | import os
  |      ^^
3 | print(calculate_price(12.5, 4))
help: Remove unused import: `os`
1 | from math_utils import calculate_price
- import os
2 | print(calculate_price(12.5, 4))
```

- **Pytest**: 自动化测试框架, 用于回归验证重构后的代码逻辑。



```
collected 1 item
test_math_utils.py . [100%]
===== 1 passed in 0.01s =====
```

1.3 实验7: 用调试器定位归并排序缺陷

1.3.1 问题描述

给定归并排序实现存在逻辑缺陷, 要求不直接重写算法, 通过调试定位问题并完成最小修复, 同时补充单元测试验证正确性。

1.3.2 缺陷代码分析

原`merge`函数的`else`分支逻辑错误: 当右数组当前元素更小时, 错误地取左数组元素并移动右指针, 直接导致右数组元素丢失, 排序结果错乱。

Listing 1: 带缺陷的merge片段

```

1 while i < len(left) and j < len(right):
2     if left[i] <= right[j]:
3         result.append(left[i]); i += 1
4     else:
5         result.append(right[i]); j += 1      %-----wrong

```

1.3.3 调试过程

1. 断点设置在merge函数入口，跟踪每次双指针比较过程 2. 观察到当‘right[j] > left[i]’时，加入结果的是更大的left[i]，且右指针移动跳过更小元素 3. 确认循环结束后剩余拼接逻辑‘result + left[i:] + right[j:]’本身正确，无需修改

```

(Pdb) p f"left[{i}]= {left[i]}, right[{j}]= {right[j]}"
'left[0]=3, right[0]=1'

```

1.3.4 修复后完整代码

Listing 2: 修复后的merge_sort.py

```

1 def merge(left, right):
2     result = []
3     i = j = 0
4     while i < len(left) and j < len(right):
5         if left[i] <= right[j]:
6             result.append(left[i])
7             i += 1
8         else:
9             result.append(right[j])
10            j += 1
11    return result + left[i:] + right[j:]
12
13 def merge_sort(a):
14     if len(a) <= 1: return a
15     m = len(a)//2
16     return merge(merge_sort(a[:m]), merge_sort(a[m:]))

```

1.3.5 测试结果

运行pytest后两类用例全部通过： - 给定随机输入排序结果与内置sorted完全一致 - 含大量重复元素的列表排序结果正确 - 空列表、单元素、逆序列表等边角场景均符合预

期

```
coolicer@1:~/q07$ pytest -v
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/coolicer/q07
collected 2 items

test_merge_sort.py::test_given_case PASSED [ 50%]
test_merge_sort.py::test_duplicate_case PASSED [100%]

===== 2 passed in 0.01s =====
```

1.3.6 核心知识点

- 双指针归并排序的核心逻辑：始终取两个数组头部的最小元素
- 调试器的断点、变量观察功能快速定位逻辑错误
- 单元测试用于回归验证重构/修复的正确性

1.4 实验8：先测量，再优化慢速词频程序

1.4.1 原慢速代码

```
1 words = open("words.txt", encoding="utf-8").read().split()
2 unique = []
3 for word in words:
4     if word not in unique: # O(n)
5         unique.append(word)
6 print("count=", len(unique))
```

1.4.2 操作流程

1. 生成数据：运行 generate.words.py 生成包含 30000 个随机词的 words.txt。
2. 基线测量：使用 time python wordfreq.py 运行 2 次，记录 real 时间并计算中位数[2](@ref)。

```
coolicer@1:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.090s
user    0m0.082s
sys     0m0.008s
coolicer@1:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.088s
user    0m0.083s
sys     0m0.004s
```

3. 性能分析：运行 `python -m cProfile -s cumulative wordfreq.py`，确认耗时集中在列表的 `__contains__` 操作上[1](@ref)。

```

coolicer@i:~/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
      1012 function calls in 0.081 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.000   0.000   0.081    0.081 {built-in method builtins.exec}
      1   0.079   0.079   0.081    0.081 wordfreq.py:1(<module>)
      1   0.002   0.002   0.002    0.002 {method 'split' of 'str' objects}
      1   0.000   0.000   0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
    1000   0.000   0.000   0.000    0.000 {method 'append' of 'list' objects}
      1   0.000   0.000   0.000    0.000 codecs.py:319(decode)
      1   0.000   0.000   0.000    0.000 {built-in method _codecs.utf_8_decode}
      1   0.000   0.000   0.000    0.000 {built-in method builtins.print}

```

4. 最小修复：引入辅助 `set` 用于快速查重，保持原逻辑输出 `count` 不变。
5. 优化后测量：再次运行 `time` 命令 2 次，计算中位数与加速比。

```

coolicer@i:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.014s
user    0m0.008s
sys     0m0.006s
coolicer@i:~/q08$ time python3 wordfreq.py
count= 1000

real    0m0.015s
user    0m0.008s
sys     0m0.006s

```

1.4.3 优化后代码

```

1 words = open("words.txt", encoding="utf-8").read().split()
2 seen = set()
3 unique = []
4 for word in words:
5     if word not in seen: # O(1)
6         seen.add(word)
7         unique.append(word)
8 print("count=", len(unique))

```

1.4.4 核心知识点

- 时间复杂度：Python 列表的成员检测为 $O(n)$ ，集合（基于哈希表）为 $O(1)$ 。
- Profiling：cProfile 的 `-s cumulative` 参数可按累计耗时排序，快速定位热点函数[4](@ref)。

- **基准测试：**使用 `time` 命令多次运行取中位数，消除系统负载波动带来的误差。

2 练习:命令行环境

2.1 参数与 Globbs

2.1.1 - 参数的作用

操作与指令：

```
1 touch -- -myfile
2 ls -l -- -myfile
3 rm -- -myfile
```

原理解释： `--` 告诉程序停止解析后面的选项（flag）。如果不加 `--`，`touch -myfile` 会被解析为传递了 `-m`、`-y` 等选项参数，从而报错。

```
coolicer@1:~$ touch -- -myfile
coolicer@1:~$ ls -- -myfile
-m myfile
coolicer@1:~$ rm ./-myfile
coolicer@1:~$ ls -- -myfile
ls: cannot access '-myfile': No such file or directory
```

2.2 ls 命令组合参数

操作与指令：

```
1 ls -lah --color=auto
```

原理解释： `-a` 显示隐藏文件，`-l` 列表格式，`-h` 易读大小，`-t` 按修改时间排序，`--color=auto` 输出颜色。

```
coolicer@1:~$ ls -lath --color=auto
total 84K
drwxr-x--- 17 coolicer coolicer 4.0K 9月 5 12:19 .
drwx----- 3 coolicer coolicer 4.0K 9月 4 20:45 snap
drwxr-xr-x 2 coolicer coolicer 4.0K 9月 4 20:45 Music
drwxr-xr-x 2 coolicer coolicer 4.0K 9月 4 20:45 Pictures
drwxr-xr-x 2 coolicer coolicer 4.0K 9月 4 20:45 Videos
drwxr-xr-x 2 coolicer coolicer 4.0K 9月 4 20:45 Documents
```

2.3 环境变量与返回码

2.3.1 marco 与 polo 函数

操作与指令： 创建 `marco.sh`：

```

1 #!/bin/bash
2 marco() { export MARCO_DIR=$(pwd); }
3 polo() { cd "$MARCO_DIR" || return; }

```

执行 `source marco.sh`。

```

coolicer@1:~$ nano marco.sh
coolicer@1:~$ source marco.sh`
> ^C
coolicer@1:~$ cd q06
coolicer@1:~/q06$ marco
coolicer@1:~/q06$ cd ~
coolicer@1:~$ polo
coolicer@1:~/q06$

```

2.4 循环调试脚本

操作与指令：

```

1 #!/bin/bash
2 count=0
3 while true; do
4     count=$((count+1))
5     output=$(bash debug.sh 2>&1)
6     if [ $? -ne 0 ]; then
7         echo "Failed after $count runs"
8         echo "$output"
9         break
10    fi
11 done

```

```

✓ 第 97 次成功
✓ 第 98 次成功
✓ 第 99 次成功
✗ 失败在第 100 次，日志已保存
----- 失败时的普通输出(stdout)： -----
Something went wrong
----- 失败时的错误输出(stderr)： -----
The error was using magic numbers

```

2.5 信号与任务控制

2.5.1 后台任务处理

操作与指令：

```

1 sleep 10000
2 # Ctrl+Z
3 bg
4 pkill -f "sleep_10000"

```

```

coolicer@1:~$ sleep 10000
^Z
[1]+  Stopped                  sleep 10000
coolicer@1:~$ bg
[1]+  sleep 10000 &
coolicer@1:~$ pgrep -af "sleep 10000"
2439 sleep 10000
coolicer@1:~$ pkill -f "sleep 10000"
[1]+  Terminated              sleep 10000
coolicer@1:~$

```

2.6 别名与点文件

2.6.1 纠错别名

操作与指令:

```

1 alias dc='cd'

```

2.7 远程机器 (SSH)

2.7.1 端口转发配置

操作与指令: 编辑 `/.ssh/config` 加入 `LocalForward 9999 localhost:8888`, 服务端运行 `python -m http.server 8888`, 本地访问 `http://localhost:9999`。

```

python3 -m http.server 8888
Serving HTTP on 0.0.0.0 port 8888 (http://0.0.0.0:8888/) ...

```

```

The key's randomart image is:
+--[ED25519 256]--+
|      . . . .      |
|      .             |
|      . . . .      |
|    . . . o .      |
|   .S . = * o      |
|    o  E B +       |
|   . + X . *       |
|   . = % + + o      |
|   . = / * B o      |
+-----[SHA256]-----+

```

3 练习: Debugging and Profiling

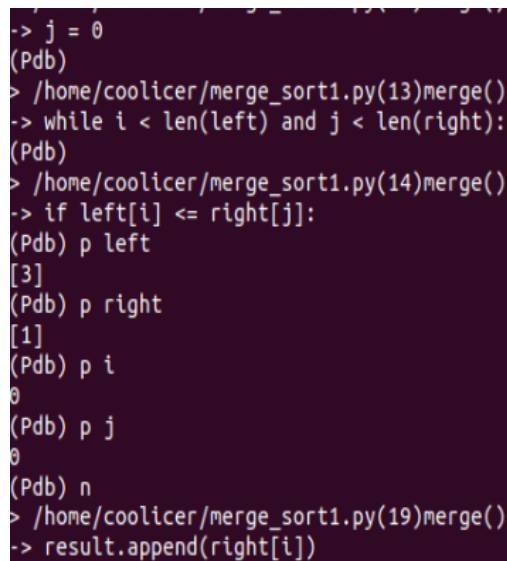
3.1 调试排序算法

3.1.1 归并排序 Bug 调试

操作与指令:

```
1 gcc -g merge_sort.c -o merge_sort
2 gdb ./merge_sort
```

原理解释: 伪代码中切片语法需注意边界, 实际实现中 `merge` 函数需正确处理数组剩余元素。



```
-> j = 0
(Pdb)
> /home/coolicer/merge_sort1.py(13)merge()
-> while i < len(left) and j < len(right):
(Pdb)
> /home/coolicer/merge_sort1.py(14)merge()
-> if left[i] <= right[j]:
(Pdb) p left
[3]
(Pdb) p right
[1]
(Pdb) p i
0
(Pdb) p j
0
(Pdb) n
3
> /home/coolicer/merge_sort1.py(19)merge()
-> result.append(right[i])
```

3.2 反向调试

3.2.1 内存损坏排查

操作与指令:

```
1 gcc -g corruption.c -o corruption
2 rr record ./corruption
3 rr replay
```

原理解释: `curve_scores` 循环越界写入, 覆盖了 `students[1].id`。

3.3 AddressSanitizer

3.3.1 内存错误检测

操作与指令:

```
1 gcc -fsanitize=address -g uaf.c -o uaf
2 ./uaf
```

原理解释： Use-After-Free 错误。free(greeting) 后再次写入和读取。修复需删除 free 后的操作。

```
coolicer@1:~$ gcc -fsanitize=address -g uaf.c -o uaf && ./uaf
Hello, world!
=====
==25029==ERROR: AddressSanitizer: heap-use-after-free on address 0x503000000040
at pc 0x5f23cbfc22ca bp 0x7ffde6627440 sp 0x7ffde6627430
WRITE of size 1 at 0x503000000040 thread T0
#0 0x5f23cbfc22c9 in main /home/coolicer/uaf.c:12
#1 0x70b2d0c29d8f in __libc_start_call_main ../sysdeps/nptl/libc_start_call_
main.h:58
#2 0x70b2d0c29e3f in __libc_start_main_impl ../csu/libc-start.c:392
#3 0x5f23cbfc2184 in _start (/home/coolicer/uaf+0x1184)
```

```
coolicer@1:~$ gcc -fsanitize=address -g uaf.c -o uaf && ./uaf
Hello, world!
Jello, world!
```

4 实验心得

在本周的实验中，我了解了什么是命令行以及如何去使用它，感受到了它强大的功能和便利的使用方式。并且学会了怎么对一个代码项目进行调试。

通过实际操作，理解了信号捕获、作业控制、SSH 转发等命令行实用技巧调试方面，体验了 gdb 断点、rr 反向调试与 AddressSanitizer 的高效定位能力。